CASTLE: Collaborative Analytics via Secure, Trustworthy and Scalable Query Evaluation

Xiling Li, Jennie Rogers
Northwestern University
{xiling.li, jennie}@northwestern.edu

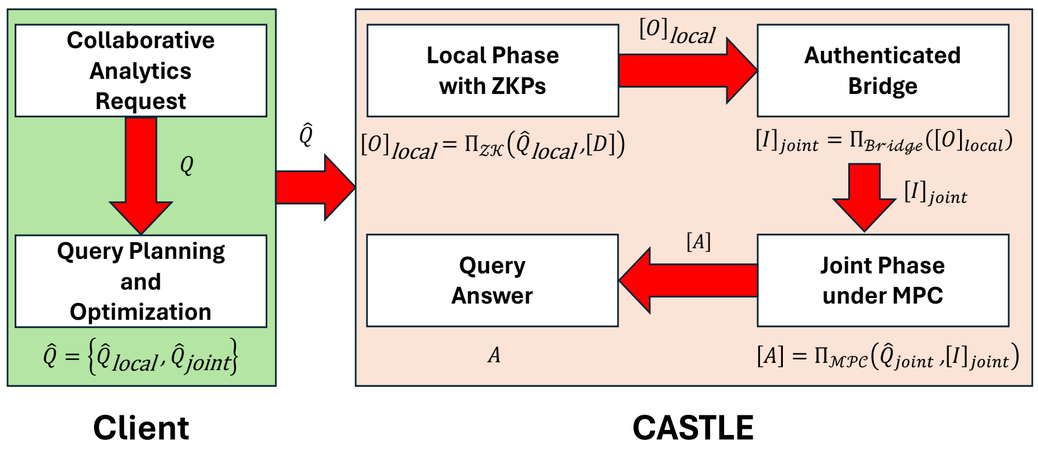
Motivation

- **Problem:** Industry and public sectors increasingly engage in collaborative data analysis, yet they require strong guarantees over their private data.
- **Approach:** A **private data federation** (PDF) with **secure multiparty computation** (MPC).
- **Status Quo:** To alleviate computationally expensive MPC-based PDFs, SOTA systems delegate per-partition computation to local plaintext execution to reduce the use of MPC, while failing to ensure the integrity of the source records.

Our Solution

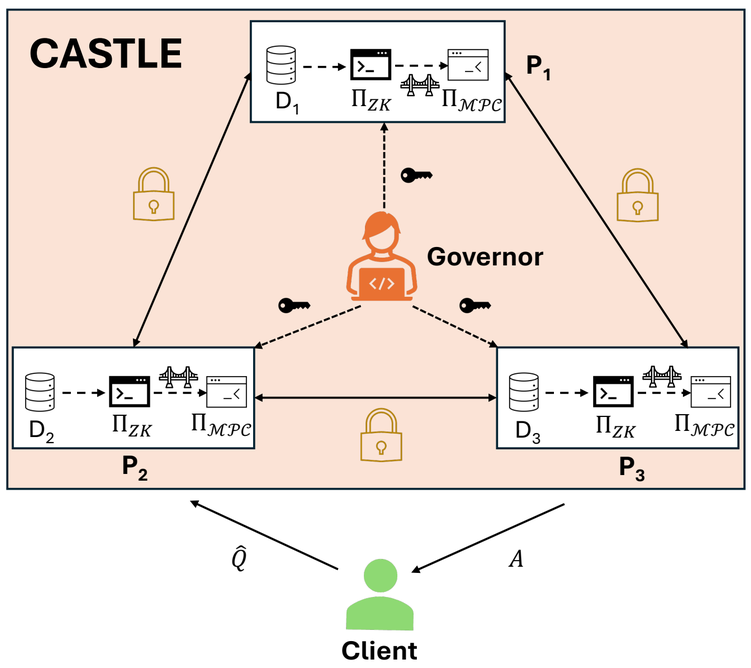
We propose CASTLE, a **C**ollaborative **A**nalytic system based on **S**ecure, **T**rustworthy, and sca**L**able query **E**valuation, splitting the system into three phases:

- It pushes operators related to each partition (e.g., filters, projections, per-partition joins) down to **a local phase** with zero-knowledge proofs (ZKPs).
- It operates the remaining operators that require the computation over all local outputs (e.g., cross-party joins and global aggregations) in **a joint phase**.
- It securely connects the two phases with **an authenticated bridge** protocol to ensure the end-to-end integrity for the whole query lifecycle.



Architecture

Consider a PDF consisting of **n data providers**, $P = \{P_i\}_{i\in[1,n]}$, and **a client**, C, with **a governor**, G, that accelerates pseudo-randomness generation for MPC:



- **Data Provider**: Each data provider i or P_i holds its private partition D_i from the union $D = \bigcup_{i=1}^{n} D_i$ and acts as a computing party in the secure and verifiable query evaluation, connecting with others via secure channels.
- **Client**: C launches collaborative analytics by distributing a query execution plan $\hat{Q}$ to computing parties and receives the final query answer A.
- **Governor**: G generates pseudo-randomness and distributes them to all $P_i \in P$.

Security Model

CASTLE prevents the malicious adversary, $\mathcal{A}$, from deviating from MPC protocols arbitrarily, where CASTLE tolerates up to $n-1$ corrupted parties.
The client, C, is semi-honest, because C faithfully generates the plan $\hat{Q}$ and distributes it to computing parties, while C may attempt to infer unauthorized data.
In addition, **the governor, G**, has the following properties:

- **Semi-Honest**: G faithfully generates and distributes randomness, but may seek to learn unauthorized information.
- **Non-Colluding**: G does not collude with any corrupted P_i.
- **Non-Client**: G neither sends the query nor observes the answer.

To support **end-to-end integrity** from the query to the answer, CASTLE ensures:

- **Correctness**: If all parties honestly follow the protocol to execute Q over D, C receives a legitimate query answer A.
- **Soundness**: If any P_i deviates from the protocol, the remaining parties including C will abort with all but negligible probability.
- **Zero Knowledge**: If $\mathcal{A}$ corrupts up to $n-1$ data providers, it learns nothing about D beyond what can be inferred from A.

To connect the local and joint phases with the end-to-end guarantee, we leverage the algebra in IT-MAC, which underlies VOLE-based ZKPs; we choose this approach because it scales efficiently to query evaluation. ZK sets achieve $O(N)$ sorts and joins.

Authenticated Bridge Protocol

Each data provider P_i, acting as a prover $\mathcal{P}_i$, works with the remaining providers, acting as verifiers $\{\mathcal{V}_j\}_{j\in[1,n],j\neq i}$. $[O_i]_{local}$ is $\mathcal{P}_i$'s verified output for a sequence of l committed bits $\{[x_h]\}_{h\in[1,l]}$. Let $\mathcal{H}$ be a hash function. Let $\oplus$ and $\cdot$ be XOR and AND ops.
Initial State. $\mathcal{P}_i$ holds each bit x_h and a MAC tag $m_{j,h}$, and each $\mathcal{V}_j$ holds a local key $k_{j,h}$ and a global key Δ_j ($m_{j,h}$, $k_{j,h}$, $\Delta_j \in \mathbb{F}_{2^\kappa}$), satisfying algebraic relationship:

$$\underbrace{m_{j,h}}_{\text{LHS}} = \underbrace{k_{j,h} \oplus x_h \cdot \Delta_j}_{\text{RHS}} \tag{1}$$

Random Mask. $\mathcal{P}_i$ generates **a random mask** $r_h \in \mathbb{F}_{2^\kappa}$ for each bit x_h.
LHS Computation. $\mathcal{P}_i$ locally performs XOR $\{m_{j,h}\}^{h\in[1,l]}_{j\in[1,n],j\neq i}$ with $\{r_h\}_{h\in[1,l]}$:

$$LHS = \bigoplus_{h=1}^{l} \left(\left(\bigoplus_{j=1,j\neq i}^{n} m_{j,h} \right) \oplus r_h \right) \tag{2}$$

LHS Hash. $\mathcal{P}_i$ locally computes **a digest** $d_{LHS} = \mathcal{H}(LHS)$.
Secret Sharing. $\mathcal{P}_i$ secret shares $\{[\hat{x}_h]\}_{h\in[1,l]}$, $\{[r_h]\}_{h\in[1,l]}$ and $[d_{LHS}]$, and all verifiers secret share $\{[k_{j,h}]\}^{h\in[1,l]}_{j\in[1,n],j\neq i}$ and $\{[\Delta_j]\}_{j\in[1,n],j\neq i}$, respectively.
RHS Computation. All parties jointly compute the $[RHS]$ and open $[RHS]$:

$$[RHS] = \bigoplus_{h=1}^{l} \left(\bigoplus_{j=1,j\neq i}^{n} [k_{j,h}] \oplus [r_h] \right) \oplus \left(\bigoplus_{j=1,j\neq i}^{n} [\Delta_j] \right) \cdot \left(\bigoplus_{h=1}^{l} [\hat{x}_h] \right) \tag{3}$$

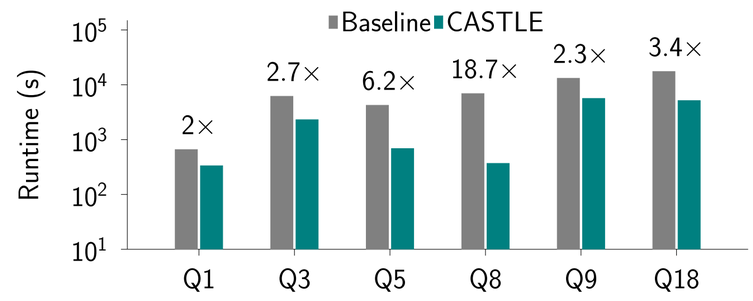
RHS Hash. Each party computes $d_{i,RHS} = \mathcal{H}(RHS)$ and secret shares its $[d_{i,RHS}]$.
Equality Check. All parties jointly compute the equality for all digests:

$$\forall i \in [1,n], [b] = ([d_{LHS}] == [d_{i,RHS}]) \tag{4}$$

Verification. All parties open $[b]$. If **b=1**, they return $[I_i]_{joint}$ including the sequence of trustworthy secret shared bits $\{[\hat{x}_h]\}_{h\in[1,l]}$. Otherwise, they abort the protocol.

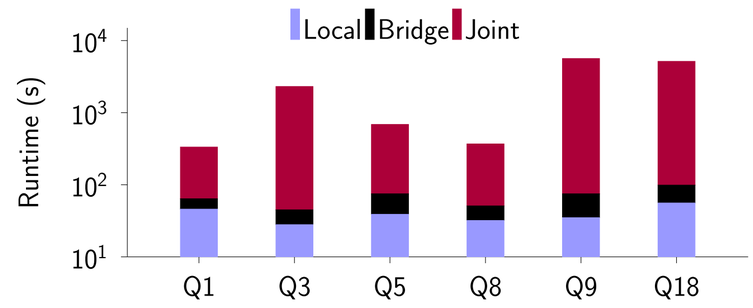
Full MPC Baseline vs. CASTLE

We evaluate the baseline and CASTLE over TPC-H benchmark with $SF = 0.01$.



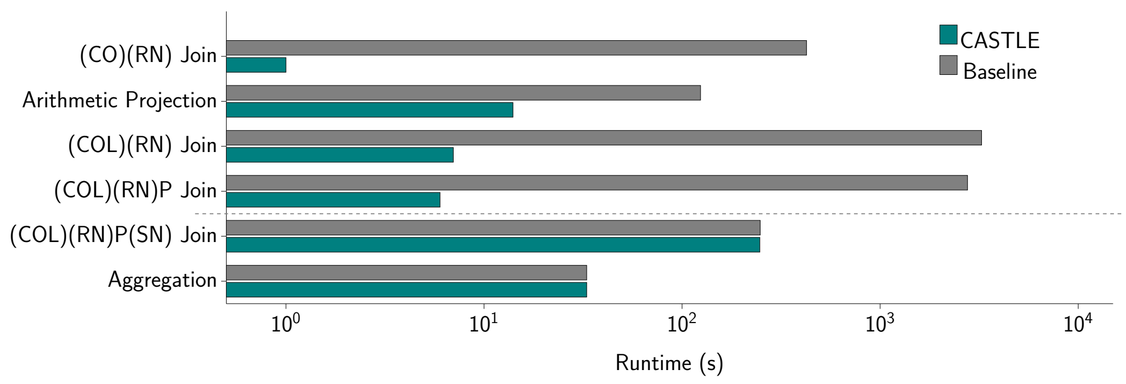
CASTLE Breakdown

We break down CASTLE into the local phase, bridge, and joint phase runtime, where the local phase runtime is the maximum across all providers, as they run concurrently.



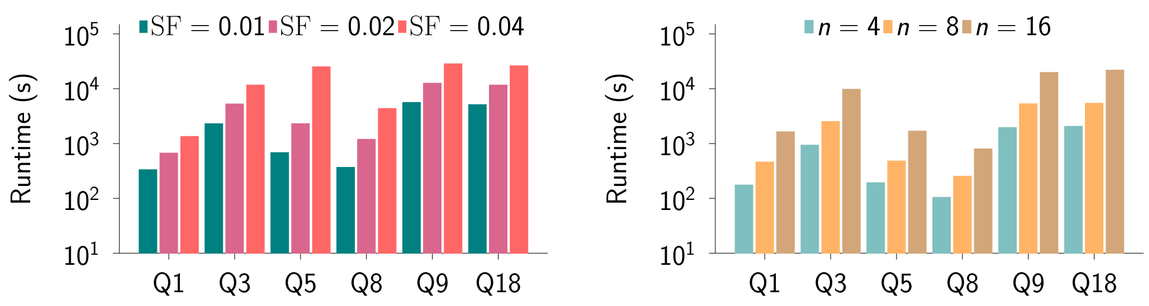
Among all queries, joint phases run roughly one to two orders of magnitude longer than their local or bridge runtime due to the expensive, unavoidable joint operators.

Q8 Operator Ablation



Due to the largest savings ($\sim 93\% \rightarrow \sim 3\%$) on local joins and one of the smallest joint-phase bottlenecks, $Q8$ achieves $18.7\times$ speedup, the largest across all queries.

Scaling with respect to data volume and provider count



As data size increases, local and bridge phases scale linearly, while the joint phase scales super-linearly, inheriting the respective $O(N \log^2 N)$ and $O(N^2)$ cost of oblivious sorts and joins, where $Q5$ scales the steepest due to the algorithm switch by circuit-aware cost model. As n grows with $\mathrm{SF} = 0.003$ fixed, local phase stays nearly constant, while the joint phase slows $8.7 - 10.7\times$ from $n = 4$ to 16, driven by MPC's linear per-bit and quadratic per-party communication costs (a network-bound behavior).